

AI-Wrevolusi

Iteration 2 Security Aspects Report

FIT5120 Industry Experience | 17 September 2026

Report overview

AI-Wrevolusi helps users explore how AI relates to their work tasks and choose learning activities. Iteration 2 adds more useful ways to save and continue this journey. It also gives the system more information to protect, including account workspaces, learning records, calendar plans and saved AI questions and answers.

The main security improvement is clearer ownership of saved work. The current code checks which account owns a workspace and rejects outdated saves. It also places limits on several AI requests and validates model responses before using them. These controls address real problems, such as one account receiving another account's delayed save or an AI service returning an unsupported result.

However, the review found gaps that still need attention. The older authentication routes return session tokens in response bodies. Logout does not immediately invalidate a copied access token. Browser caches can hold personal work, and an enabled fallback can send task information to an external AI service even when the main API key is missing. Some security-related tests also need repair before they can provide reliable assurance.

Overall assessment: the current checkout has useful security controls, but this review does not establish that the hosted system is ready for unrestricted use with sensitive workplace information. The first priorities are session handling, explicit control over external AI processing, safe logout, and reliable security tests.

What the report provides

Sections 1 to 3 explain the changes, system boundaries and information at risk. Sections 4 to 9 examine the controls and their limits. Sections 10 to 13 give the risk register, test results, proposed improvements and response procedures. Section 14 explains the main lessons. The appendices identify the code and checks behind the findings.

Evidence snapshot: Iteration 1 release 2499736 compared with current checkout 57d0d78 on kuan/design-prototype. This is a source and local-test assessment. Hosted settings, live database permissions and real provider behaviour were not verified.

1. What changed from Iteration 1

The comparison uses the saved iteration-1-release tag, which resolves to commit 2499736 dated 3 September 2026. The current checkout is commit 57d0d78. The release is on a different line of Git history, so this is a comparison of two snapshots rather than a claim that every intervening commit is part of one straight release sequence. [C1]

The earlier snapshot already contains password hashing, cookie sessions, refresh-token records and protected backend resource routes. These are carried-forward controls. They should not be presented as new achievements in Iteration 2.

Area	Iteration 1 release	Current Iteration 2	Security effect
Accounts	Email-based backend authentication already existed.	Username accounts support the main user journey without requiring a real email address.	Less contact information is requested, but account recovery is limited.
Saved work	The frontend kept much of the profile journey in browser storage.	Account workspaces synchronize selected profile, learning and planning data with the server.	Ownership and conflicts now need checks on both the browser and server.
Learning	No current learning-progress or daily-brief models in the release snapshot.	Courses, chapter progress, check-ins and daily briefs have dedicated storage.	More personal activity must be protected against disclosure and incorrect updates.
AI integration	Exposure matching used local reference-based processing.	New AI routes and configurable external providers support matching and assistance.	Task text can cross a third-party boundary. Abuse and misleading output need controls.
Task Assist	No saved Task Assist interaction model.	One question and answer can be saved per user and task key.	Ownership, duplicate requests and retention become important.
Shared state	No current account workspace synchronization layer.	Owner checks, revision checks and account-switch guards protect saves.	These reduce accidental mixing and loss of user work.

The main architectural shape remains a React frontend, a FastAPI backend and PostgreSQL. The configured Vercel frontend/backend service split also existed in the release. The security change comes mainly from the broader data flows and stored records, rather than a completely new hosting design. [C1, C2]

2. How information moves through the system

A trust boundary is a point where information moves between systems with different control. For example, the browser belongs to the user, while the backend decides whether a request is allowed. We should validate information again when it crosses that boundary.

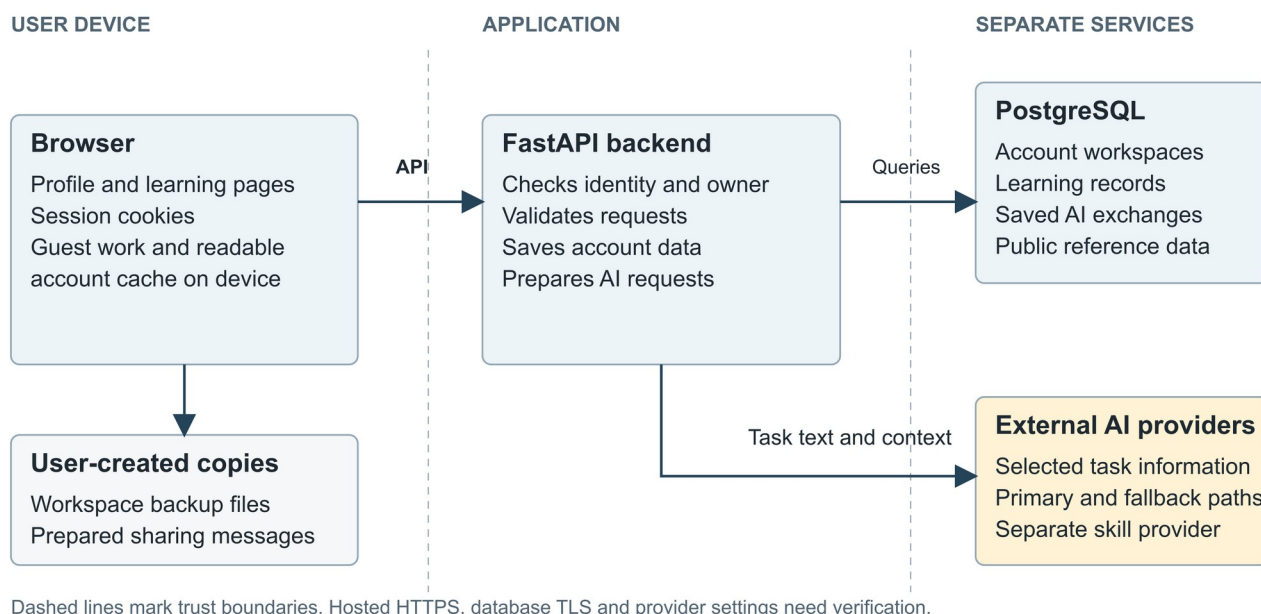


Figure 1. Source-level view of the current system. Arrows show information exchange. The hosted transport settings still need verification. [C2, C3, C7, C8]

Browser to backend

The frontend sends requests to `/api/v1` and includes session cookies. The backend must check identity, ownership and input rules. A hidden button or page guard only changes what the interface shows. It cannot stop someone from sending a request directly. [C2, C4]

Backend to database

The backend reads and writes account workspaces and other user records. The database also stores public reference information. These records have different sensitivity, even when they are held in the same database. Database credentials and permissions therefore matter as much as page-level checks. [C3, C9]

Backend to external AI services

Some operations can send task descriptions, notes, questions or candidate information to a configured provider. Task Assist sends the saved task context and the user's question. A separate skill-direction service also has its own provider configuration. The report cannot infer either provider's retention policy from application code. [C7, C8]

Browser to files and external links

The user can download an unsynced workspace backup. Planning code can also prepare an event message for WhatsApp. These features move information outside account storage. The message link is a sharing action initiated by the user, not proof that a message has been sent. [C3, C13]

3. What needs protection and why

Not all information has the same risk. Public occupation and course records need protection mainly against incorrect changes. A user's task descriptions, plans and questions also need protection against unwanted disclosure.

Information or function	Where it appears	Harm if exposed or changed
Passwords and sessions	Password hashes, cookies and refresh-token records	Another person may gain account access.
Account workspace	Server JSON and browser cache	Work tasks, learning choices and plans may be disclosed or overwritten.
Learning activity	Progress, check-ins and daily briefs	Someone may infer a person's routine or alter their learning history.
Task Assist exchanges	Saved task text, notes, question and reply	A user may accidentally preserve confidential workplace details.
Calendar and sharing content	Planner entries, local backups and prepared messages	Event titles and availability may leave the application.
Reference and course data	Reference tables, catalogue and import files	Incorrect data may lead to misleading assessment or learning suggestions.
AI and database credentials	Backend environment settings	Disclosure could allow unauthorized service use or database access.

These assets are visible in the current account, learning and Task Assist models, workspace keys and planning code. This is an inventory of source-defined information, not a count of records in the hosted database. [C3, C8, C9, C13]

Security goals

Confidentiality means that saved personal work should only be available to its owner and the services that need it. Removing a real email field helps reduce collection, but a detailed work description can still identify a person or employer.

Integrity means that saved work and assessment results remain accurate. A late request must not overwrite another account's work, and a generated suggestion must not be treated as verified source data.

Availability means that the core journey remains usable when saving or AI generation fails. A helpful fallback should be clear about its limits and should not silently delete user choices.

Accountability means that the team can investigate important actions without recording passwords, tokens or unnecessary task details. A proposed event log needs a clear purpose, access rules and retention period.

Responsible use means that AI exposure and learning suggestions support the user's judgement. They should not be presented as a prediction that a particular person will lose their job.

4. Accounts and session security

What is already present

The password helper uses `pwdlib` with its recommended password-hashing configuration. The authentication service issues a short-lived access token and a longer-lived refresh token. The configured defaults are 30 minutes and 14 days. The refresh record stores a token identifier, expiry and revocation time. It does not store a hash of the complete refresh token. [C4]

The cookie helper applies `HttpOnly`, which prevents direct JavaScript access through the cookie API. `SameSite` defaults to `lax`. The `Secure` flag is configurable and defaults to `false` for local development. These settings need to be checked on the hosted site before claiming that production cookies are protected correctly. [C4, C10]

Username registration accepts 3 to 32 letters, numbers or underscores and passwords of 8 to 128 characters. Usernames are normalized to lowercase. The new login and registration responses return account details rather than the token pair. The interface also tells users that email recovery is unavailable. However, the older email registration schemas still accept an empty password. Password rules need to cover every mounted registration path. [C3, C4, C5]

Token handling still needs improvement

The older `/auth/login` and `/auth/refresh` routes return both tokens in JSON as well as setting cookies. The current frontend uses the refresh route. This means the cookie flag alone does not keep all token values away from same-origin JavaScript. If malicious script runs in the page, the response body is another possible access path. The proposed change is to make browser session endpoints return only the account or status information the frontend needs. [C4, C5]

Logout revokes the presented refresh token and clears the browser cookies. It does not immediately invalidate an access token that was copied earlier. The access-token check validates the token and loads the user, without checking a session-revocation record. It also does not check `is_active` at that point. We should define how logout and account disabling affect existing sessions, then test that policy. [C4]

Refresh rotation reads the record, checks it and then marks it revoked. The source does not make that consumption a single locked or conditional operation. Two overlapping refresh requests are therefore a race concern to test in PostgreSQL. This review did not reproduce that race against a database.

Why these changes matter

Hashing helps protect passwords after a database leak. It does not stop someone trying passwords stolen from another website. Similarly, `HttpOnly` limits cookie reads but does not stop malicious page script from acting through an authenticated browser. These controls need login throttling, safe rendering and a clear session policy around them. [W1]

5. Ownership and the integrity of saved work

Account workspaces

The workspace route first resolves the account from the authenticated user. A save must name that same owner. The backend then locks the account row, checks the submitted revision and increments the revision after a successful update. A wrong owner or stale revision returns a conflict response. This is a practical improvement because two tabs should not silently overwrite each other's changes. [C3]

The browser also tracks the active workspace session. A response from an older account session cannot update the new account's revision. Shared learning operations check the session again after waiting for course information. The focused frontend tests passed for account switching, delayed saves, guest separation and rollback after storage failure. These checks provide local evidence of those specific behaviours. [C3, T1]

Guest work and account work

Guest records remain in browser storage. Signed-in records use the account workspace and an account-specific cache. The interface offers an import choice during authentication. That choice is initially selected, so we should make the effect especially clear on a shared computer. A future improvement is to show which items will be imported and let the user choose them. [C3, C5]

The latest shared-state changes preserve learning records when the work profile changes and ask the user to review their plan. Practice-only updates also preserve saved learning. This supports integrity because a routine edit should not silently remove a user's earlier decisions. It does not prove that those decisions are still suitable, which is why the review notice matters. [C6, T1]

Learning and Task Assist records

Learning routes use the authenticated user when reading and writing progress, check-ins and daily briefs. Task Assist records are scoped by user and task key. The model includes a unique constraint on that pair and checks for valid interaction states. A stored task key is a bounded string, not a foreign key to the older task table. [C8, C9]

These are useful ownership controls. However, several current Task Assist tests fail because their fixtures still use the older UUID contract. The failing checks include isolation and concurrent requests. We should repair and rerun them before treating those behaviours as fully verified. [T2]

A public write path needs review

The mounted POST `/api/v1/occupations` route has no authentication dependency and passes new records to its repository. This carried-forward path writes the older occupations model, not the separate official `ref_occupations` table. It should be removed from public access or protected with a specific administrator permission. No live write was attempted. [C14]

A saved record is not a complete audit trail

The older SQL schema contains `review_events`, but this review found no current runtime path that writes every profile correction to that table. Workspace revisions detect stale updates. They do not preserve a full history of who changed each field. Task Assist state rules also do not make records impossible for a database administrator to alter. A minimal security event log remains proposed work. [C12]

6. Input, browser and request protection

Validate the specific rules we need

Pydantic schemas check the rules defined for each request. For example, Task Assist limits task text to 4,000 characters, notes and questions to 2,000 characters, and registration batches to 50 details. Its schemas reject extra fields. Workspace updates allow a defined set of keys and check that values contain valid JSON. [C3, C8]

The workspace limit compares the length of serialized JSON with 2,000,000 characters. This is a post-parsing application check, not proof of a request-body byte limit at the server edge. Most workspace values also receive only a JSON-syntax check, while the shortlist has additional shape validation. We should add field-specific limits where a malformed value could break later reads. [C3]

The inspected database paths use SQLAlchemy expressions and bound values. This keeps user input separate from SQL command structure. TypeScript helps developers catch frontend mistakes, but a person can bypass those types by making an API request directly. Input validation and query binding solve different problems and both are needed.

Render generated text safely

The Task Assist interface inserts the question and reply as ordinary React text. It does not interpret the reply as HTML in the inspected component. This reduces a direct route for cross-site scripting, or XSS, where untrusted text becomes executable page content. It does not prove that the entire application is free from XSS. [C8]

Course links open with noopener noreferrer, which limits interaction with the original browser tab. Course URLs should also be checked against allowed schemes and intended providers before being published. The tab protection alone does not prove that every course destination is trustworthy. [C13]

Cross-site requests need their own defence

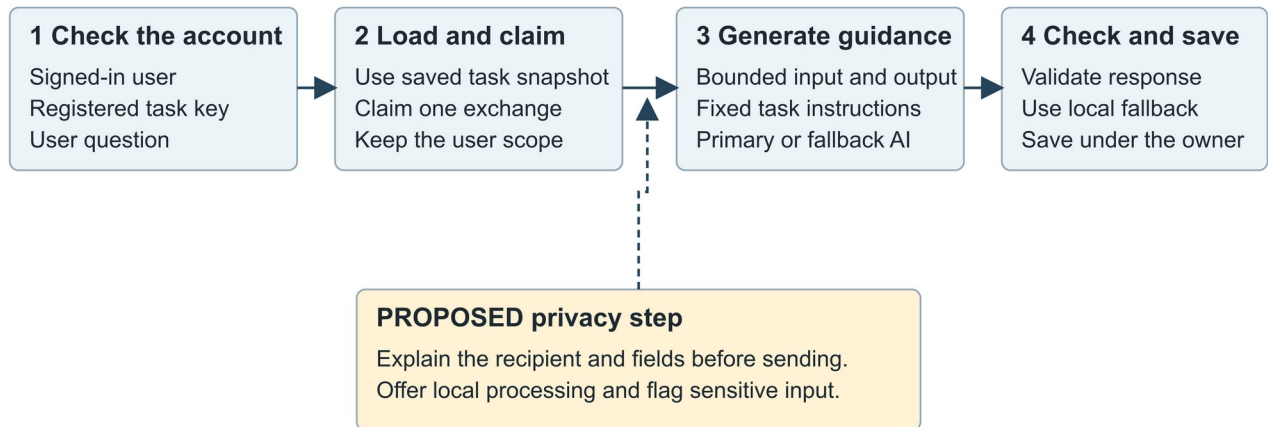
Cross-site request forgery, or CSRF, happens when a different site causes the browser to send an unwanted authenticated request. The code uses a CORS origin list and SameSite=Lax cookies. These help, but no dedicated CSRF token or Origin-checking policy was found in the inspected request paths. CORS mainly controls browser access to cross-origin responses. It is not user authorization. [C2, C4]

We propose a consistent server-side CSRF defence for cookie-authenticated changes, supported by Origin checks and tests for allowed and rejected requests. We also need to confirm that read-only routes do not make unexpected changes. OWASP describes SameSite as an additional layer rather than a universal replacement for CSRF protection. [W2]

No application-defined Content Security Policy was found in the inspected application and deployment configuration. A tested policy should be added to limit script and resource origins. Hosted response headers were not checked, so this is a source-configuration gap rather than a confirmed missing live header. [C2, C10]

7. AI controls and their limits

The AI integration is now part of the security assessment. Task text is untrusted input. It may contain an ordinary work description, private information, or instructions intended to change how the model behaves.



A schema checks response shape. It does not prove factual accuracy or stop every prompt attack.

The proposed privacy step is not an implemented control.

Figure 2. Current Task Assist flow and the additional privacy check proposed before an external request. The proposed check is not implemented. [C7, C8]

Controls visible in the current code

Task Assist uses a fixed system prompt that treats the request fields as data. It tells the model to stay within the task, avoid claims about job loss and keep important decisions with a person. The request uses the server's registered task snapshot, while the answer request supplies a task key and question. [C8]

The response must match a defined schema. Its reply has a 1,200-character limit. The prompt separately asks for fewer than 180 words, but the word limit is a model instruction rather than the same hard schema rule. Pattern checks reject some high-risk claims and likely prompt or credential disclosure. Failed or rejected generation can produce clearly labelled local guidance. [C8]

Task Assist requests use a 20-second timeout per provider attempt, no configured request retries and no response cache for that operation. A fallback attempt can still add time, so this is not a guaranteed total request deadline. One saved exchange per task key also does not create a total per-account quota across new keys. [C7, C8]

What these controls cannot guarantee

Prompt instructions and pattern checks cannot guarantee that every harmful or misleading reply will be caught. Valid JSON can still contain bad advice. Human review, minimal information sharing and tests with deliberately misleading inputs remain necessary. OWASP recommends multiple layers around untrusted model input and output. [W3]

The skill finalizer has a specific weakness: when no returned evidence phrase passes its exact-text check, it can keep the original unverified skills. An isolated probe confirmed this helper behaviour. It should reject unsupported evidence or use a verified local result. Also, constraining an answer to a caller-supplied candidate list does not prove that the list contains official reference records. [C7, T3]

8. Privacy and responsible AI use

External processing needs a clear user choice

The configuration enables an OpenCode fallback by default. When the main provider is unavailable or no main key is configured, eligible requests may still reach an external service. Missing credentials therefore do not prove that AI processing is local. The separate skill-direction provider also needs its own review. [C7]

Task Assist sends the task text, notes and question. Even without a username or email in that payload, a user could type a customer name, internal project or confidential workplace process. The inspected flow does not include a dependable personal-data redaction step or a clear provider-specific consent record. We should explain what leaves the application before the first external request and offer a local option. [C8]

A practical next step is a warning beside the input asking users to remove real names, customer details, passwords and confidential material. Automated detection can help flag obvious sensitive text, but it should not be described as complete anonymization. The team must also confirm the chosen providers and their handling of submitted information.

Browser storage and logout

The account cache stores workspace data as readable JSON in localStorage. Account-specific keys help the application keep work separate, but they do not encrypt the data or protect it from other code running on the same origin. Someone with access to the browser profile may also read it. OWASP warns against treating browser storage as a confidentiality boundary. [C3, W4]

Successful logout removes workspace and account-cache keys. However, the current logout function waits for a workspace save before calling the logout API or clearing local data. A save failure can therefore stop the logout sequence. We propose a separate, clear option to end the local session even when syncing fails, with an explicit warning about unsaved work and honest status if server revocation cannot be reached. [C5]

Retention and sharing

The Task Assist interface describes the exchange as permanently saved. The model stores the task context and answer, while no user-facing account deletion or Task Assist deletion route was found in the inspected runtime. A foreign-key cascade is not a complete deletion feature. We need a retention policy and an authenticated deletion process covering server records and browser copies. [C8, C12]

Downloaded workspace backups and prepared sharing messages need a preview and a short explanation of their contents. Once a user saves or shares them elsewhere, application access checks no longer protect that copy. We should avoid adding unnecessary identifiers and confirm what the user intends to share. [C5, C13]

Keep suggestions in context

The code includes wording that separates task exposure from job replacement and asks for human confirmation of AI assistance. We should keep those limits visible. Future evaluation should use varied occupations and wording, including Malaysian workplace examples, and record weak coverage. This review did not establish fairness or model accuracy across user groups. [C7, C8]

9. Deployment, secrets and ongoing protection

Configuration is a security control

The backend reads database and AI credentials from environment settings. Local environment files are ignored, and the tracked environment-related files inspected are examples or tooling files. This does not establish that Git history is free of secrets. No full-history secret scan was run. [C10]

The configuration contains a development signing-secret default and sets `cookie_secure` to false by default. There is no production startup rule in that settings class that rejects these values. We should fail startup in a production environment when the signing key is unsafe or secure cookies are disabled. This is a configuration risk, not evidence that the deployed site currently uses those values. [C10]

The database URL helper adapts SSL-related parameters for the driver. The actual database connection settings, certificate verification, database role permissions and hosted backup policy were not inspected. The report therefore makes no claim that all database traffic or backups have been verified as secure. [C10]

Request limits need to work across the service

The AI provider has an in-process request limit, with a configured default of 60 requests per minute. It is attached to provider processing rather than being a shared per-user or per-IP limit for the whole application. Multiple server processes or provider instances can have separate counters. Several public AI matching routes also have no authentication dependency. The separate skill-theme and goal calls bypass this shared provider guard. [C7]

Guest matching is useful, so the answer is not automatically to remove it. A proportionate improvement is a shared anonymous quota, tighter limits for expensive operations and per-account controls for saved assistance. Authentication attempts also need throttling. An asynchronous server or a scalable database does not provide these controls by itself.

The provider response cache is bounded, but the higher-level gateway cache is a dictionary with no eviction limit shown. Some occupation context fields also lack an explicit size cap. Both need bounds to reduce memory and request-processing risk. [C7]

Dependencies, logs and recovery

The project has frontend and backend lockfiles. These help reproduce dependency versions, but they do not prove that the versions are free of known vulnerabilities. No dependency advisory scan was run during this assessment. No checked-in GitHub Actions workflow was found in this checkout. This does not establish whether external platform checks or branch protections exist. [C10]

Useful security logs should record event type, time, result and a request identifier. They should avoid passwords, tokens, whole workspaces and full AI prompts. The team should define who can see those logs and when they are deleted. A complete security event trail was not verified in the current implementation.

Backups should be tested through a restore into a separate environment. Public reference files can help rebuild reference data, but they cannot restore personal learning records. Recovery time, backup coverage and data-loss tolerance remain decisions to confirm and test.

10. Updated risk register

These ratings are review judgements for the current source, not measured attack probabilities. They describe the remaining exposure after the controls observed in code. Hosted controls could change the ratings once verified. No risk is marked as formally accepted by the team.

Likelihood runs from 1, rare, to 5, almost certain. Impact runs from 1, negligible disruption, to 5, broad account or data compromise. The score is likelihood multiplied by impact. Scores 1 to 4 are Low, 5 to 9 Medium, 10 to 16 High and 17 to 25 Critical.

Account, session and privacy risks

ID and risk	L × I	Current protection and remaining concern	Proposed treatment
R1 Unsafe production settings	2 × 5 = 10 High	Environment settings exist, but a development signing key and insecure-cookie default are allowed. Hosted values are unverified. The impact could affect many accounts.	Reject unsafe production settings and verify actual cookie and transport behaviour.
R2 Session token exposure or reuse	3 × 4 = 12 High	Hashing, expiry and refresh rotation help. Token JSON responses, access-token lifetime after logout and the refresh race concern remain.	Remove token response bodies, define revocation behaviour and test atomic refresh.
R3 Weak passwords or repeated login attempts	3 × 4 = 12 High	Password hashes protect stored credentials. Legacy signup permits empty passwords, and no application-wide login throttle was found.	Enforce consistent password rules, shared throttling and safe monitoring.
R4 Browser data exposure	3 × 3 = 9 Medium	Account keys and successful logout cleanup help. Readable caches remain while signed in, and a failed save can block logout. Shared devices increase exposure.	Minimize persistent caches and provide a reliable local sign-out path.
R5 Sensitive text sent to AI	3 × 4 = 12 High	Bounded payloads and task-only prompts reduce scope. Free text can contain private details, and external fallback is enabled by default.	Require an approved provider policy, clear disclosure and a local option.

R1 is rated with lower likelihood because the deployed settings are unknown, not because the impact would be small. R2 and R5 are higher priorities because the source shows concrete paths where session or task information can leave the intended boundary. [C4, C7, C8, C10]

10.1 Data, request and operational risks

ID and risk	L × I	Current protection and remaining concern	Proposed treatment
R6 Wrong-account access or overwrite	2 × 4 = 8 Medium	Owner and revision checks exist. Frontend account-switch checks pass, but real-database and some Task Assist isolation evidence is incomplete.	Keep the controls and complete two-account and concurrent-request tests.
R7 Forged or malicious browser requests	3 × 3 = 9 Medium	SameSite, CORS and ordinary React text rendering help. Dedicated CSRF checks and an application-defined CSP remain gaps in the reviewed configuration.	Add and test a consistent CSRF policy and a suitable CSP.
R8 Misleading AI or reference results	3 × 3 = 9 Medium	Candidate constraints, response schemas and fallback labels help. Valid output can still mislead, and three AI contract checks fail.	Align the contract and tests, then evaluate misuse cases and weak coverage.
R9 Excess requests or AI usage	4 × 3 = 12 High	Some limits and timeouts exist. Public routes and per-process provider counters do not form a shared service quota.	Add shared quotas and monitor rejected or expensive requests.
R10 Long retention or uncontrolled copies	3 × 3 = 9 Medium	Some records are user-scoped. Permanent exchanges, caches and downloaded files lack a complete verified deletion lifecycle.	Define retention and deletion, with previews for export and sharing.
R11 Weak detection or recovery	3 × 4 = 12 High	Source history and lockfiles help investigation. A complete event trail, current advisory scan and restore drill were not verified.	Add essential monitoring, dependency checks and a tested recovery procedure.
R12 Public occupation writes	3 × 3 = 9 Medium	The older occupation creation route has no app authentication check. It affects the legacy occupation table, with hosted exposure unverified.	Remove the public write or require administrator permission.

R6 remains Medium because the reviewed ownership controls are meaningful, while assurance is incomplete. R9 is High because a person can automate requests without following the intended page flow. The rating does not claim that a load attack was performed. [C3, C7, T1, T2]

How to use this register

The treatment strategy is mainly to reduce risk through specific controls. If an external AI provider has not been approved for the information involved, the safer temporary treatment is to disable that external path and provide a local result. Cloud-provider arrangements do not transfer our responsibility for choosing what information to send.

Backend, frontend and deployment responsibilities are proposed in Section 12. They are roles for the team to assign, not invented commitments by named members. Ratings should be reviewed after each acceptance check, especially when hosted settings or database tests provide new evidence.

11. Security checks and findings

The review combined Git comparison, current-source inspection and selected existing tests. Tests used local stubs or mocks where provided by the repository. No real AI provider requests, production penetration tests or live database mutation tests were used for this assessment.

Check	Observed result	What the result supports
Git baseline and source comparison	Release and current commit identified. Core authentication files are unchanged.	Distinguishes carried-forward controls from Iteration 2 changes.
Frontend ownership and related checks	33 passed, 0 failed across four files.	Local account separation, save behaviour, planning rules and Task Assist UI checks.
Selected backend checks	104 passed, 9 failed across eleven files.	Some account, learning, AI validation and persistence checks pass. The selected suite is not clean.
Configuration and route inspection	Concrete controls and gaps documented in Sections 4 to 9.	Source-level evidence only, with no claim about active hosting settings.
Live cookies, headers, TLS and database controls	Not run.	No hosted assurance is claimed.
Dependency and full-history secret scans	Not run.	Lockfiles and ignored files alone are not security scan results.

What failed in the backend checks

One AI contract check expects a maximum of two skill matches while the current published contract permits three. Two skill-route checks still expect provider or fallback behaviour that differs from the current local matching path. These failures need a decision about the intended behaviour, followed by aligned tests and implementation. They should not simply be removed. [T2]

Six Task Assist checks use assumptions from the older server-task UUID contract. The current API and model use string task keys. The failures cover task identification, saved exchanges, repeated requests, account isolation, concurrent requests and pending-state recovery. Several fixtures fail before they can demonstrate the behaviour named in the test. [T2]

This does not prove six separate live vulnerabilities. It does mean that the current test run does not verify those protections. Updated fixtures must still exercise the security purpose, including rejection of another user's data and only one accepted concurrent claim.

Limits of the evidence

Mock-based tests do not prove that PostgreSQL row locks, constraints and competing transactions behave correctly in the deployed setup. Browser helper tests also do not replace real browser checks for cookies, XSS or CSRF. The next validation stage should use a disposable database and two controlled accounts before testing hosted transport settings.

12. Proposed improvements and justification

The following work is proposed. It was not implemented during this report review. The order follows the risks above and focuses on changes that can be checked clearly.

Priority 1: before collecting sensitive real-user information

Session responses and account rules. The backend owner should remove raw tokens from browser-facing JSON responses, apply consistent password rules and decide how logout, refresh reuse and account disabling end sessions. Make refresh consumption atomic. Acceptance requires no token fields in responses, rejection of empty passwords and disabled users, the chosen logout behaviour, and a PostgreSQL test where only one concurrent refresh succeeds. Addresses R2 and R3.

Production configuration checks. The deployment owner should require a strong signing key and secure cookies in production, then record actual response headers and database transport settings. This prevents a developer-friendly default from becoming a production setting. Acceptance requires startup rejection of unsafe values and a hosted check of cookie flags, HTTPS behaviour and the configured database connection. Addresses R1.

External AI policy. The backend and product owners should list each allowed provider and fallback, confirm what fields are sent and explain that processing to users. Disable unapproved fallback paths. Acceptance requires an outbound-request capture showing the intended destination and fields, a tested local option, and clear disclosure before external processing. No real confidential data is needed for this test. Addresses R5.

Reliable logout. The frontend owner should allow a user to leave a shared device even when a save fails. Offer a clear unsaved-work choice and separate local cleanup from the status of server revocation. Acceptance requires a failed-save scenario in which the user can still clear local work and exit the account view, with an honest message if server logout is unavailable. Addresses R4.

Repair security test coverage. The backend and test owners should resolve the nine failures against the agreed current contracts. Acceptance requires those scenarios to pass without removing ownership or concurrency assertions. Then run two-account and concurrent-request tests against disposable PostgreSQL. Addresses R6 and R8 and improves confidence in R2.

Close unsupported write and evidence paths. The backend owner should protect or remove public occupation creation and make skill finalization reject unsupported evidence. Acceptance requires an anonymous write to fail without creating a record and invalid evidence phrases to be discarded. Addresses R8 and R12.

Why these come first

These actions address information disclosure, session access and missing evidence in existing features. Adding more AI features before resolving them would increase the amount of user information handled without answering the current security questions.

12.1 Next iteration and release checks

Proposed action and role	Reason	Evidence needed to close the action
Shared request limits. Backend and deployment owners.	In-memory provider counters do not cover all callers or server instances.	Login and AI quotas hold across two workers. Excess requests receive a controlled response without an external AI call.
CSRF and browser headers. Backend and frontend owners.	Cookie authentication and browser script loading need explicit rules.	Legitimate changes work. Untrusted-origin and missing-token cases fail. CSP is tested against the actual frontend resources.
Privacy lifecycle. Product and backend owners.	Permanent exchanges and local copies need a defined purpose and end point.	A documented retention policy, authenticated deletion and checks that server records and relevant browser copies are removed. Backup handling is explained.
Safe backups and sharing. Frontend owner.	Downloaded and shared information leaves account protection.	Users can inspect the content first. Unnecessary personal fields are absent and cancelled actions do not export or share.
Security events and recovery. Backend and deployment owners.	Incidents need evidence and a working way to restore service.	Events record useful metadata without secrets. A backup is restored in a separate environment and the outcome is recorded.
Dependency and secret checks. Repository maintainer.	Reproducible dependencies can still contain known vulnerabilities.	Advisory and secret scans run for the release revision. Findings have a recorded fix or explicit team decision.

Longer-term work

Account recovery needs a deliberate design because the current account journey does not request a real email address. Options may include recovery codes or another verified channel. We should choose a method after weighing the extra personal information and support work it creates. The team should not imply that recovery exists before it is implemented.

Multi-factor authentication can be considered for higher-risk accounts or a wider deployment. It is not a substitute for the session and authorization fixes already identified. Likewise, a complex audit ledger should not be the first response to missing audit events. A small, useful event trail with restricted access is a more practical starting point.

AI evaluation should include unclear tasks, conflicting instructions, sensitive-text examples and unsupported career claims. Record failures by task type and wording. Candidate limits and response schemas should remain in place while these tests improve the quality of the content.

The team can assign named owners and dates when planning the next iteration. This report sets the order and completion checks without inventing a schedule or a risk acceptance decision.

13. Proposed incident response and recovery

These procedures are recommendations for the next iteration. A tested incident process was not verified during this review.

Suspected credential or signing-key exposure

The deployment owner should disable the affected credential and replace it in the correct environment. Check where it was used and when the exposure may have started. If the signing key is affected, invalidate the relevant sessions through the agreed session design and require users to sign in again. Review logs without copying secret values into the incident notes.

Recovery is complete only after the old credential is rejected, the new configuration works and the cause of exposure has been addressed. A new key alone does not explain what an attacker may already have accessed.

Suspected access to another user's work

Temporarily restrict the affected operation, preserve relevant request identifiers and reproduce the problem using two controlled accounts. Identify which records and time period may be affected. Correct the ownership check and add a regression test for the exact case. Re-enable the operation after the check passes and the team has decided what communication is needed.

Unexpected AI output or data transfer

Disable the affected external provider path, including its fallback, and use a clear local or unavailable response. Preserve only the minimum information needed to understand the failure. Review the destination, submitted fields and output checks. Re-enable the feature after a focused evaluation shows the specific problem is addressed. Any required user communication should be decided from the confirmed facts and applicable obligations.

Database outage or lost records

Stop conflicting writes where necessary and show a clear save status. Do not silently replace unsynced work with an empty workspace. Restore into a separate environment first, verify the record relationships and inspect whether recent changes are missing. Reference datasets can be reimported, but personal progress needs a suitable backup.

After recovery

Record what happened, what information was affected, how it was contained and which test or monitoring rule will catch it earlier next time. Assign the follow-up work and update the relevant risk rating. This keeps the incident review focused on preventing a repeat rather than only restoring the page.

14. Reflection and overall assessment

Iteration 2 makes the application more useful because people can save learning choices, return to their plans and receive task-specific assistance. The same changes make security more than a login problem. The system now needs to manage personal information across the browser, the database, exported files and external AI services.

The strongest improvement shown by the review is the clearer ownership of saved work. Server-side owner and revision checks are supported by passing frontend tests for delayed saves and account changes. The AI code also contains concrete limits, structured responses and fallback behaviour. These are practical controls worth keeping.

The main lesson is to describe each control at the level the evidence supports. A cookie flag does not prove that tokens never reach JavaScript. A successful local test does not prove that database concurrency works. A model response that matches a schema is not automatically correct, and a feature described as local may still have an external fallback.

For the next iteration, we should first close the session and privacy gaps and restore reliable security tests. We should then verify the hosted settings, request limits and recovery process. This gives the team a clearer basis for deciding what information the system is ready to handle.

Assessment boundary: this report supports targeted improvement of the inspected checkout. It is not a certification, a completed penetration test or a claim of legal compliance. Risk acceptance and the move to broader real-user use still require the team's decision based on the checks described here.

Appendix A. Project evidence

The references below identify primary project evidence. Paths are relative to the repository root. Line numbers refer to the source inspected on 17 September 2026 and may change after edits. The main code paths reviewed had no local modifications at the time of inspection. Existing documentation edits were left untouched.

ID	Source and what it supports
C1	Git tag iteration-1-release → 2499736fb8da7f42f579b734dbc34220566b8d81. Current HEAD → 57d0d782fd9ff4e26f14113b924ac537eb800e3e. Snapshot comparisons of auth files, models, routers and frontend storage.
C2	vercel.json, lines 1–33. backend/app/main.py, lines 24–66. frontend/src/services/api.ts, lines 28–73. Service routing, mounted APIs, CORS and credentialed browser requests.
C3	backend/app/routers/accounts.py, lines 18–106. frontend/src/services/accountStorage.ts, lines 4–200. Account input, workspace limits, owner/revision checks, browser caches and account-switch guards.
C4	backend/app/core/security.py, lines 10–96. backend/app/services/auth.py, lines 43–117. backend/app/routers/auth.py, lines 25–44. backend/app/models/refresh_token.py. backend/app/schemas/auth.py, lines 4–12. backend/app/schemas/user.py, lines 6–9. backend/app/core/config.py, lines 32–41. Hashing, token lifetime, identifier storage, rotation, revocation and token JSON responses.
C5	frontend/src/components/account/AccountProvider.tsx, lines 24–116 and 133–165. frontend/src/components/account/AuthDialog.tsx, lines 25–74 and 100–118. frontend/src/services/api.ts, lines 59–61. Import choice, recovery wording, backup download, refresh use and logout order.
C6	frontend/src/features/learning-planning/ and frontend/src/features/work-profile/. frontend/tests/learningOwnership.test.mjs. Shared learning updates, saved-work preservation and current account ownership.
C7	backend/app/core/config.py, lines 48–77. backend/app/services/ai_gateway.py. backend/app/routers/ai.py, lines 46–235. backend/app/services/skill_directions.py, backend/app/services/learning_goal.py, lines 17–29, and backend/app/routers/skill_directions.py, lines 16–37. Provider selection, external fallback, request counters, public matching routes and separate skill provider.
C8	backend/app/schemas/task_assist.py, lines 10–54. backend/app/services/task_assist.py, lines 8–98. backend/app/services/task_assist_records.py. backend/app/models/task_assist.py, lines 17–55. backend/app/routers/ai.py, Task Assist routes. frontend/src/pages/Analysis/components/TaskAssistDialog.tsx, lines 145–205.

Appendix A continued

ID	Source and what it supports
C9	backend/app/routers/learning.py, backend/app/services/learning.py, backend/app/services/learning_records.py and learning-related models. Catalogue validation and user-scoped progress, check-ins and daily briefs.
C10	backend/app/core/config.py, lines 14–46 and 86–98. backend/app/db/session.py, lines 8–19. .gitignore, frontend/package-lock.json, backend/uv.lock, frontend/package.json and backend/pyproject.toml. Defaults, database setup and dependency configuration. Tracked-file listing checked environment-related filenames.
C11	Selected tests listed in Appendix B. These are the source of the reported local results, not evidence of hosted behaviour.
C12	db/schema.sql, review_events definition. Targeted searches of backend/app for review-event writes, session-wide revocation and user-facing deletion routes. No complete runtime audit or account-deletion lifecycle was found in the inspected paths. This is a bounded source finding.
C13	frontend/src/features/learning-planning/planModel.ts, lines 105–109. frontend/src/pages/LearningCentre/components/CourseDetailDrawer.tsx, lines 59–66. Prepared sharing text and external course-link attributes.
C14	backend/app/routers/occupations.py, lines 23–29. backend/app/services/occupations.py, lines 17–19. backend/app/repositories/occupations.py and backend/app/models/occupation.py. Unauthenticated creation of legacy occupation records.

How the source review was performed

The repository graph was used to locate relevant code. Its coverage was checked, then the findings were confirmed against current files because some graph snippets reflected older file lengths. Configurations, tests excluded from the graph and Git snapshots were read directly. Negative findings are limited to the inspected routes and configuration, not every possible hosted control.

Official guidance used for recommendations

- **W1. OWASP Session Management Cheat Sheet.** Used for the limits of cookie flags and the need for a clear session lifecycle.
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- **W2. OWASP Cross-Site Request Forgery Prevention Cheat Sheet.** Used for SameSite limits and server-side CSRF controls. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- **W3. OWASP LLM Prompt Injection Prevention Cheat Sheet.** Used for layered handling of untrusted AI input and output.
https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
- **W4. OWASP HTML5 Security Cheat Sheet.** Used for browser-storage confidentiality limits.
https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

All four pages were checked on 17 September 2026. They guide the proposed controls. They do not prove that this project has implemented them.

Appendix B. Local test record

T1. Frontend checks

Run from the frontend directory using the bundled Node runtime:

```
node --test tests/accountStorage.test.mjs tests/learningOwnership.test.mjs tests/learningPlanning.test.mjs tests/taskAssist.test.mjs
```

Result: 33 passed, 0 failed. The run covered workspace ownership, conflicts, late saves, account switching, guest import boundaries, rollback after storage failure, planning rules and Task Assist interface/state checks. Some tests inspect source contracts while others execute helpers with mocked storage and requests. This was not a full browser security test.

T2. Backend checks

The selected backend run used the existing Python 3.12 environment and local test doubles.

Result: 104 passed, 9 failed. From the repository root, `backend/.venv/bin/python -m pytest` ran the following files under `backend/tests/`, with `-q`:

```
test_auth.py, test_accounts.py, test_ai_api_contract.py, test_ai_gateway.py, test_ai_provider.py, test_ai_provider_fallback.py, test_ai_provider_responses.py, test_task_assist.py, test_task_assist_persistence.py, test_learning_endpoints.py, test_learning_persistence.py.
```

Test setup disabled external AI fallback and automatic table creation. Endpoint and provider tests used dependency replacements or HTTP mock transports. No live database or provider was used.

The nine failed checks were:

1. `test_published_and_runtime_output_limits_are_bounded`
2. `test_skill_route_uses_a_safe_gateway_fallback_for_malformed_provider_json`
3. `test_skill_provider_evidence_must_be_an_exact_task_substring`
4. `test_task_assist_answer_requires_a_server_owned_task_uuid`
5. `test_registered_detail_is_answered_from_the_server_snapshot_and_saved`
6. `test_second_question_returns_the_permanent_first_exchange_without_calling_ai`
7. `test_same_detail_is_isolated_by_authenticated_user`
8. `test_concurrent_requests_grant_only_one_provider_call`
9. `test_pending_state_recovers_to_saved_fallback_without_a_second_provider_call`

These failures remain open in this report. The assessment did not change application code or rewrite tests to obtain a passing result. No real-provider quality test, hosted-header check, dependency advisory scan or live PostgreSQL concurrency test is included in these counts.

T3. Additional isolated checks

A separate offline probe confirmed that the legacy login returns both token fields while cookies remain `HttpOnly`, the shared authentication dependency accepts an inactive mocked user, both legacy registration schemas accept an empty password, and the skill finalizer can retain nonmatching evidence. These are six observations outside the pytest count. They establish local code behaviour, not a live exploit or evidence of an actual breach.